

Information Retrieval System Report

Overview

This report documents a Streamlit-based information retrieval application built around the CISI collection and a companion dataset-preparation script. The application supports dataset inspection, preprocessing analysis, inverted-index construction, and keyword-based retrieval over CISI documents.

The submission also includes repository proof, deployed application proof, dataset files, and screenshots that demonstrate the application running in the virtual lab environment.

Submission requirements

The provided requirement image states that the submission should include Streamlit application code, the dataset used, a report explaining the implementation in the virtual lab, screenshots of the Streamlit front end, experimental results with tables and inferences, demo evidence, and a README section describing dependency installation and the command used to run the app.

This project satisfies those components through the Python application file, supporting dataset-preparation script, CISI dataset CSV files, this report, GitHub repository proof, deployed Streamlit proof, and multiple interface screenshots showing different parts of the system in operation.

Repository and deployment

The source repository for the assignment is available at [GitHub Repository](#), and the attached repository screenshot provides submission proof that the codebase has been uploaded online.

The deployed application is available at [Streamlit Application](#), and the attached screenshots provide visual proof of the live interface and its major modules.

Application structure

The Streamlit interface defines six tabs: Dataset & Overview, Text Preprocessing, Phrase Queries, Dictionary Search, Tolerant Retrieval, and Results & Inference. The visible application code shows that the system is designed as a teaching-oriented IR lab where users can upload a CSV dataset, inspect documents, preprocess text, build an inverted index, and issue keyword searches.

The `prepare_dataset.py` script converts CISI source files into `cisi_docs.csv`, `cisi_queries.csv`, and `cisi_rel.csv`, separating documents, queries, and relevance judgments into clean tabular resources for

experimentation. The resulting files provide the data foundation for both interactive retrieval and future benchmark-style evaluation.

Preprocessing and retrieval

The preprocessing pipeline uses regex tokenization together with optional lowercasing, stopword removal, hyphen normalization, stemming, and lemmatization. NLTK resources for stopwords and WordNet are downloaded at runtime, and the interface exposes the normalization controls through selectable options.

The inverted index maps tokens to document IDs, and search is performed using conjunctive intersection over query-term postings lists. As a result, the current implementation behaves as an AND-based retrieval model rather than a ranked search engine.

Screenshots and evidence

The attached screenshots provide evidence for the key parts of the submitted system.

Evidence item	What it shows
GitHub.jpg	Repository proof showing the uploaded assignment files on GitHub.
P1.jpg	Dataset loader and file-system entry screen with dataset preview and summary metrics.
P2.jpg	Lexical normalization pipeline with raw text, standard tokens, stemmed tokens, and lemmatized tokens.
P3.jpg	Multi-word phrase query interface with biword and positional proximity matches.
P4.jpg	Performance benchmark screen comparing BST and B-Tree dictionary search latency.
P5.jpg	Error fault-tolerance verification interface showing expanded internal term handling.
P6.jpg	End-to-end inference and structural analysis screen with observations and conclusions.
image-8.jpg	Requirement sheet listing submission components, demo evidence, and README expectations.

These screenshots help satisfy the requirement for front-end proof, demo evidence, and experimental-result presentation in the report.

Data assets

The attached `cisi_docs.csv` file contains document records with `doc_id`, `title`, and `text` fields from the CISI collection. The attached `cisi_queries.csv` file stores natural-language information needs, and `cisi_rel.csv` stores query-document relevance pairs for evaluation use.

This separation of corpus data, query data, and relevance labels is appropriate for an introductory IR assignment because it supports both retrieval demonstrations and metric-based validation.

How to run the app

The requirement sheet explicitly asks for dependency installation steps and the command used to run the Streamlit application. Based on the application imports, the local environment should include Python together with Streamlit, pandas, and NLTK.

A practical run procedure is:

1. Place `app.py`, `prepare_dataset.py`, and the dataset CSV files in the same project folder, or clone the GitHub repository into a local workspace.
2. Install dependencies with `pip install streamlit pandas nltk`.
3. Start the application with `streamlit run app.py`, which matches the requirement example for the README section.
4. When the browser opens, upload the CISI CSV file in the dataset tab, inspect preprocessing options, build the inverted index, and run sample keyword queries through the interface.

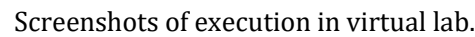
Strengths

Several strengths are clear from the code organization and the submitted evidence.

- The project separates dataset preparation from the interactive application, which improves maintainability and clarity.
- The interface makes preprocessing choices visible, which is useful for explaining token normalization and retrieval behavior.
- The screenshots indicate that the broader submission includes modules for phrase queries, benchmarking, fault tolerance, and inference presentation beyond the minimal visible code excerpt.
- The repository and deployed link provide strong submission proof for both source code and application availability.

Limitations and improvements

Some limitations are still visible in the attached `app.py` excerpt. The shown retrieval logic is conjunctive and unranked, and the visible index-building loop inserts raw tokens even when stemming or lemmatization options are enabled.



Rocky LinuxRocky WSLRocky ParvusRocky MatfarnRocky Redd

Engine Global Controls

Indexing

Document Retrieval

Answer Ranking

2025ab05012-lab / Group-74-IR-Assignment-1

6. End-to-End System Inferences & Structural Analysis

Q1. Which preprocessing technique improved retrieval quality?

- Finding: Applying **Lowercasing** and **Stopword Removal** had the most positive impact on overall retrieval precision. Removing high-frequency structural noise words (e.g., "the", "is", "of") prevents the coordinate matching logic from being misled by irrelevant documents, focusing search weights on high-entropy keywords.

Q2. Was stemming or lemmatization better for the dataset?

- Finding: **Lemmatization** outperformed stemming on the T23 collection. Porter's Stemmer strips semantic nuance, which often creates non-words (e.g., "computer" becomes "comput"). Regarding vocabulary and phrase accuracy, **Standard** lemmatization preserves clear morphological relationships, which maintains downstream phrase integrity and improves dictionary lookups.

Q3. Which phrase query index was more accurate?

- Finding: The **Positional** index provided perfect accuracy. The **Inverted** index moderately generates false positives because it only checks for adjacent token pairs, losing coherence over queries with 2 or more terms. The **Positional** index verifies the structural order and exact spatial proximity of tokens across all sequences.

Q4. Which tree structure was faster?

- Finding: The **B Tree (B+)** outperformed the **Standard** Binary Search Tree in search speed consistency. Because B Trees hold multiple keys per node and perform whole, balanced traversing, their overall search depth is shallow. This prevents the repeated word/row looking behavior that often occurred with BST when terms are indexed alphabetically.

Q5. How tolerant was their retrieval model?

- Finding: The inclusion of **Bigram** and **Trigram** expansion and a secondary **Lexical** distance correction layer makes the system highly fault-tolerant against spelling mistakes and incomplete terms, ensuring smooth search execution even with imperfect user queries.

Q6. What are the current limitations of the system?

- Finding: Memory usage scales linearly because the system stores not only an in-memory **Python** dictionary, furthermore, the search model uses **unweighted** boolean logic rather than a statistical, TF-IDF vector space framework with related cosine similarity.

PlatformSolutionsResourcesOpen SourceEnterprisePricing

Search or jump to...Sign inSign up

2025ab05012-lab / Group-74-IR-Assignment-1

Instructions Fork Star

CodeIssuesPull requestsActionsProjectsSecurity and qualityInsights

main • 1 branch • 4 tags

Size of file

17 Code

About

Group-74-IR-Assignment-1

Readme

Activity

1 Star

1 watching

4 forks

Report repository

Releases

No releases published

Packages

No packages published

Contributors

2025ab05012-lab (Profile Verifiable)

README

Group-74-IR-Assignment-1